

ML for Health

Laboratory # 3: segmentation of moles

Prof. Monica Visintin - Politecnico di Torino

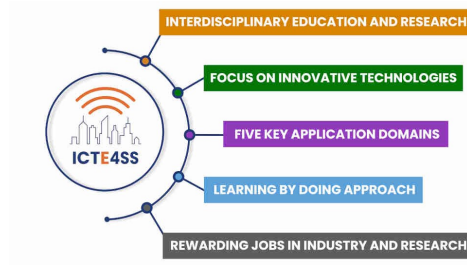


Table of Contents

Laboratory # 3: description

The goal

The idea

The details

TO DO

Description [1]

- ▶ Download file `images.zip` from the DropBox folder for laboratory 3.
- ▶ Unzip the file, you'll get several jpeg images of moles, the name of the files are
 1. `low_risk_n.jpg` (where `n` is an integer) for 11 moles that have a low probability of being melanoma (i.e. tumors)
 2. `medium_risk_n.jpg` (where `n` is an integer) for 16 moles that have a low probability of being melanoma
 3. `melanoma_n.jpg` (where `n` is an integer) for 27 moles that have a high probability of being melanoma
- ▶ View all the pictures, so that you have an idea of what you have to work with.
- ▶ The goal is to help dermatologists in the classification of moles.

Table of Contents

Laboratory # 3: description

The goal

The idea

The details

TO DO

Goal of the lab

- ▶ We want to **segment** the moles, i.e. to find the region of the picture in which the moles are present. At the end of the lab, you must be able to find the **contour of the mole**.
- ▶ Having segmented the moles, it is then possible to extract the features (i.e. the numbers) that are relevant for the diagnosis of melanoma.
- ▶ Dermatologists use 5 features (ABCDE):
 - A asymmetry
 - B border
 - C color
 - D diameter
 - E evolution
- ▶ For example a feature that can be extracted is the ratio between the length of the border/contour of the mole and the circumference of a perfect circle with the same area: the higher this ratio, the higher the indentation of the mole border, the higher the risk of the mole to be a cancer.

Table of Contents

Laboratory # 3: description

The goal

The idea

The details

TO DO

Main idea

- ▶ We use **K-Means** in scikit-learn to find **three color clusters**, i.e. we quantize of the image so that it has only three colours. It is possible to use more than 3 colors or 2 colors, but 3 is fine.
- ▶ We find the contour of the darkest cluster, corresponding to the mole.
- ▶ Unfortunately this method sometimes finds non just the mole (in the center of the picture, typically), but other regions where for example you simply have shadows in the picture.
- ▶ Then we work on the coordinates of the pixels in the the darkest cluster, and we use **DBSCAN** on these coordinates, in order to **group together dark pixels that are spatially closer**. DBSCAN will generate some clusters (with labels from 0 to $N - 1$) and will separate isolated dark pixels (outliers, with label -1). The number of found clusters depend on the chosen hyper-parameters for DBSCAN.
- ▶ The mole will probably be in the cluster with a relatively large number of points (how many? you must decide) and with a low momentum of inertia.
- ▶ Then we filter the cluster that only includes the mole, to have a smoother contour, and we apply the Sobel filter to get the mole border/contour.

Table of Contents

Laboratory # 3: description

The goal

The idea

The details

TO DO

The original image [1]

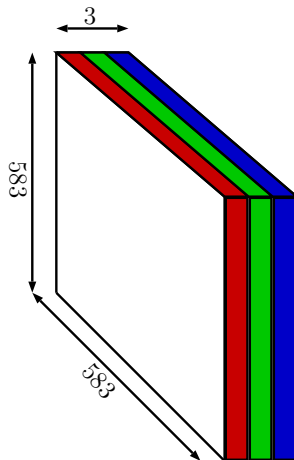
- ▶ The jpeg image is an image that has been compressed according to the jpeg standard
- ▶ To read the image in Python:

```
1 import matplotlib.image as mpimg
2 filein = ...
3 im_or = mpimg.imread( filein )
```

- ▶ `im_or` is an Nddarray with shape **N1 x N2 x 3** and elements of type `uint8` (unsigned integer with 8 bits); the original image is made of `N1 x N2` pixels (for example 583x583)
- ▶ `im_or[:, :, 0]` stores the amount of **red** color, from 0 to 255
- ▶ `im_or[:, :, 1]` stores the amount of **green** color, from 0 to 255
- ▶ `im_or[:, :, 2]` stores the amount of **blue** color, from 0 to 255
- ▶ Value `[0, 0, 0]` corresponds to black, value `[255, 255, 255]` corresponds to white

The original image [2]

The image dimension is 3, the Narray `im_or` shape is (583,583,3); `im_or[i,k,n]` stores an 8-bit unsigned integer number (from 0 to 255), for $i, k \in [0, 582]$, $n \in [0, 2]$.



The original image [3]

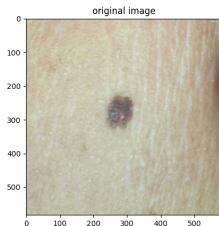
- To show the image in Python:

```
1 plt.figure()  
2 plt.imshow(im, interpolation=None)  
3 plt.title('original image')  
4 plt.show()
```

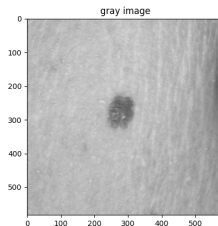
- To simplify the analysis, we convert the original color image into a grayscale image:

```
1 gray_image = np.mean(im_or, axis=2).astype(np.uint8) # convert to grayscale
```

original colors



quantized colors



The original image [4]

- ▶ `gray_image` has shape **$N_1 \times N_2$** (so it is a 2D array); `gray_image[i,k]` (the pixel in position `[i,k]`) is an unsigned integer represented with 8 bits ('uint8'), i.e. an integer between 0 and 255.

K-Means to quantize grayscale values [1]

- ▶ Import K-Means by writing:

```
1 from sklearn.cluster import KMeans
```

- ▶ To instantiate the K-means object write:

```
1 kmeans = KMeans(n_clusters=3, random_state=0)
```

- ▶ What should be the input of KMeans, since we want to quantize the grayscale colors?

K-Means to quantize grayscale values [2]

- ▶ We must convert the 2D array `gray_image` into a 1D array that simply stores the values of the grayscale level (position of the pixel is irrelevant):

```
1 N1,N2=gray_image.shape
2 im_1D = gray_image.reshape((N1*N2,1))# 1 column vector
```

- ▶ Then, use K-means as:

```
1 kmeans.fit(im_1D)
```

- ▶ The attributes of class `KMeans` are:

```
1 kmeans.cluster_centers_ # the centroids of the clusters
2 kmeans.labels_ # the N1*N2 classes/clusters each pixel belongs to
3 Ncluster = len(kmeans.cluster_centers_)
4 # Warning: it is possible that the found clusters is less than required
```

Note that the centroids are floating point numbers, but we need `uint8` numbers to show the image; therefore the centroids become:

```
1 kmeans_centroids=kmeans.cluster_centers_.astype('uint8')
```

K-Means to quantize grayscale values [3]

These three centroids represent the three colors that K-means found as representatives of all the image colors (the original image has potentially 256 different colors, but we want only 3 different colors, quantized)

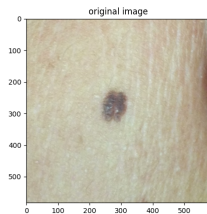
- To see the image with quantized colors, you can use the following lines:

```
1 im_1D_quant = im_1D.copy()
2 for kc in range(Ncluster):
3     im_1D_quant[(kmeans.labels_==kc),:] = kmeans.centroids[kc,:]  
4 im_quant=im_1D_quant.reshape((N1,N2))
```

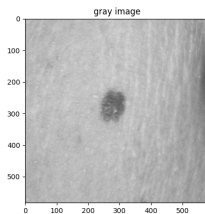
substitute the centroid value in

K-Means to quantize grayscale values [4]

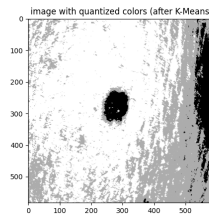
original colors



low_risk_4.jpg
grayscale colors



quantized colors



Segmentation [1]

- ▶ The classical algorithm to find the contour is the “**snake**” algorithm (or active contour), available both in Matlab and Python. **You are NOT allowed to use the snake algorithm in this lab** (otherwise you have nothing to do....)
- ▶ Among the centroids, find the darkest color, and get the positions of the pixels that belong to the cluster with the darkest color:

```

1 centroids=kmeans.centroids
2 i_col=centroids.argmax() # darkest color corresponds to minimum grayscale value
3 # 2: define the 2D-array im_clust where in position i,j you have the index of
4 # the cluster pixel i,j belongs to
5 im_clust=kmeans.labels_.reshape(N1,N2)
6 # 3: find the positions i,j where im_clust is equal to i_col (cluster with the darkest color)
7 # the 2D Narray mole_pos stores the coordinates i,j only of the pixels
8 # in cluster i_col
9 mole_pos=np.argwhere(im_clust==i_col) # Narray with two columns, storing the index [i,j]
10 # of the dark pixels

```

`im_clust` is a 2D image/array ($N1$ rows and $N2$ columns) and each element in the array takes value in the set $\{0, 1, 2\}$. Array `mole_pos` is a 2D array with as many rows as the number of pixels belonging to cluster `i_col` and 2 columns (index of the row and of the column of each pixel in the cluster).

Segmentation [2]

- ▶ You can see that in slide 15, there are two distinct regions where the quantized color is black, one is the mole, the other one is a shadow.
- ▶ We use DBSCAN to find pixels with the darkest color that are spatially close together. Use appropriate parameters so that you can identify the mole alone:

```
1 from sklearn.cluster import DBSCAN
2 clusters = DBSCAN(eps=?, min_samples=?).fit(what?)
```

- ▶ **What should be the input of DBSCAN?**

Segmentation [3]

- The input of DBSCAN must be a 2D array that stores the x and y coordinates of the pixels with the darkest color.

```
1 # find the positions i,j where im_clust is equal to i_col (cluster with the darkest color)
2 # the 2D Nddarray mole_pos stores the coordinates i,j only of the pixels
3 # in cluster i_col
4 mole_pos=np.argwhere(im_clust==i_col) # Nddarray with two columns, storing the index [i,j]
5 # of the dark pixels
```

mole_pos is the input of DBSCAN:

```
1 clusters = DBSCAN(eps=?,min_samples=?).fit(mole_pos)
```

You must correctly select `eps` and `min_samples`, the hyper-parameters of DBSCAN.

- Remember that you will probably get a cluster with label -1, which corresponds to dark isolated pixels (noise). Of course that cluster will not contain the mole. The number of clusters found by DBSCAN depends on the image and on the selected hyper-parameters. The hyper-parameters must be found so that DBSCAN correctly works on all the images of the dataset.

Segmentation [4]

- ▶ To select the correct cluster (the one corresponding to the mole) among those found by DBSCAN, consider that:
 1. In any case the mole has a compact shape, even if indented. Can momentum of inertia be a reasonable parameter to check?
 2. The mole cannot have less than 1000 pixels (approximately 30 pixels by 30 pixels). This allows you to exclude very small moles that sometimes appear together with a larger mole in the same picture. Of course we are interested in the larger mole.

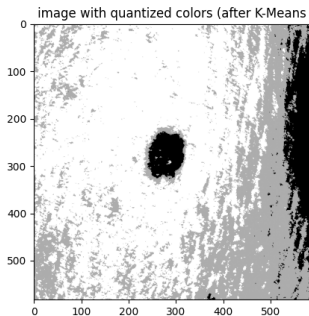
Segmentation [5]

- Once you find the correct index of the DBSCAN cluster that contains the mole (let us say that the index is `i_mole`), you can see the mole (with the original colors or in grayscale) isolated from the background using the following lines

```
1 true_mole_pos = mole_pos[clusters.labels_==i_mole] # pixel indexes [i,j] in the cluster
2 # that corresponds to the mole
3 im_only_mole_gray = 0*gray_image-1# white image
4 x=true_mole_pos[:,0]
5 y=true_mole_pos[:,1]
6 im_only_mole_gray[x,y]=gray_image[x,y]
7 plt.figure()
8 plt.imshow(im_only_mole_gray,cmap='gray',interpolation=None)
9 plt.title('original size image, segmented grayscale mole (after DBSCAN)')
10 im_mole_pos = np.ones((N1,N2),dtype='uint8')*255# white image
11 im_mole_pos[x,y]=0 # black where the mole is present
12 plt.figure()
13 plt.imshow(im_mole_pos,cmap='gray',interpolation=None)
14 plt.title('original size image, mole position')
```

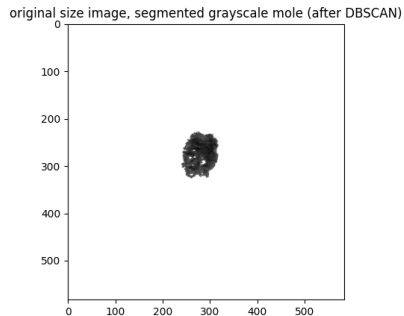
Segmentation [6]

Quantized color image



low_risk_4.jpg

Estimated position of the mole



Note that the image on the right does not have a smooth contour, and has white pixels in the middle of the black area (effects of light when the picture was taken, etc.).

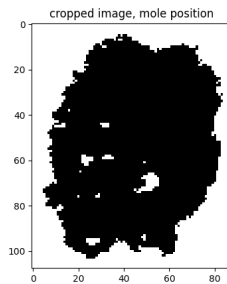
Segmentation [7]

- It is convenient to crop the image to work on a smaller image (use a margin around the found mole area):

```
1 margin = 5 # pixels around the mole (used for smoothing the image)
2 min_x = np.min(true_mole_pos[:,0]) - margin
3 max_x = np.max(true_mole_pos[:,0]) + margin
4 min_y = np.min(true_mole_pos[:,1]) - margin
5 max_y = np.max(true_mole_pos[:,1]) + margin
6 im_cropped_gray_red = im_only_mole_gray[min_x:max_x, min_y:max_y]
7 im_cropped_col = im_or[min_x:max_x, min_y:max_y, :]
8 im_cropped_mole_pos = im_mole_pos[min_x:max_x, min_y:max_y]
9 plt.figure()
10 plt.imshow(im_cropped_mole_pos, cmap='gray', interpolation=None)
11 plt.title('cropped image, mole position')
```

Segmentation [8]

Cropped image



Segmentation [9]

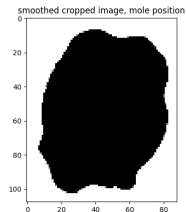
- ▶ The borders of the found area are not clear (too indented). It is better to smooth these borders, using a low-pass filter. Read the file with the prerequisites on filters in DropBox.
- ▶ Generate the new image `im_cropped_mole_pos_filt` by setting the value of the pixel in position $[i, k]$ with the **median of the values** in the area $[i - \delta : i + \delta, k - \delta : k + \delta]$ in the image `im_cropped_mole_pos`. Choose a correct value for δ .

```

1 N1,N2=im_cropped_mole_pos.shape
2 im_cropped_mole_pos_filt = 0*im_cropped_mole_pos+255# white image
3 delta = ??
4 for kr in range(delta,N1-delta):
5     for kc in range(delta,N2-delta):
6         sub = im_cropped_mole_pos[kr-delta:kr+delta,kc-delta:kc+delta]
7         im_cropped_mole_pos_filt[kr,kc] = np.median(sub)
8 plt.figure()
9 plt.imshow(im_cropped_mole_pos_filt,cmap='gray',interpolation=None)
10 plt.title('smoothed cropped image, mole position')
```

Segmentation [10]

Low-pass filtered cropped image



Contour/border [1]

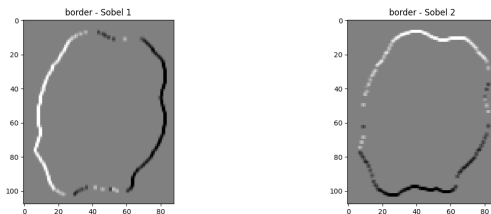
- Once you get a clean filtered image, you can use Sobel filters (both of them) to find the border of the mole. In practice each 3×3 pixel original sub-image with center pixel in position $[k_r, k_c]$ is multiplied elementwise by the 3×3 matrix $\mathbf{H}_x$ (or $\mathbf{H}_y$), and the sum of all the products is the value in position $[k_r, k_c]$ of the filtered image. The code is similar to that in slide 24.

$$\mathbf{H}_x = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 0 & -2 \\ 1 & 0 & -1 \end{bmatrix}, \quad \mathbf{H}_y = \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} = \mathbf{H}_x^T$$

Write the lines that apply Sobel filters to `im_cropped_mole_pos_filt`. Note that `im_cropped_mole_pos_filt` stores `uint8` values, first convert to integers, so that the Sobel filtered 2D array can have floating point numbers (positive and negative, since some values of $\mathbf{H}_x$ and $\mathbf{H}_y$ are negative).

Contour/border [2]

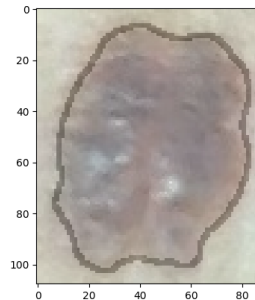
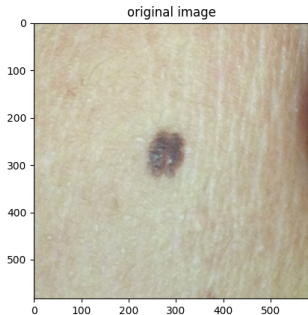
Outputs of Sobel filters



- Write the lines that generate the contour of the mole (one image), combining the outputs of the two Sobel filters.

Contour/border [3]

Contour



- Note that any measurement instrument has a precision and an accuracy. Perfect instruments do not exist, you must always find a compromise. In the picture of the previous slide you might start discussing whether the mole pixels are all included in the border or not, but the discussion might go on forever, for each pixel...

From segmentation to features

What could you do to extract relevant features from the isolated mole and its contour, according to the ABCDE rule? You are not asked to implement the code to extract the features, but you should think about it, so that during the exam you can answer the questions.

Table of Contents

Laboratory # 3: description

The goal

The idea

The details

TO DO

To do

1. Download the Python script `lab3_moles_for_students.py` from DropBox
2. Download the zipped file `moles.zip` from DropBox; unzip it as subfolder `moles` of your working folder.
3. Run `lab3_moles_for_students.py`.
4. Set a value to all the missing hyper-parameters.
5. Write the lines to implement Sobel filtering.
6. Write the lines to get the border.
7. Plot the original cropped image (with colors) and the border in the same figure (as in slide 28 on the right).
8. Set the hyper-parameters so that your script works on all the 54 images (you must check all of them; at the exam you will be asked to run your code on an image taken at random by the professor).